

Teoria de POO. Parte A: conceitos que você já usa

Este material não ensina sintaxe nova. Ele dá nome ao que você já escreveu no RPG. Cada conceito importante de Orientação a Objetos já aparece no seu código. Até agora você usou esses conceitos sem o nome técnico.

Objetivo deste material: ao final, você reconhece cada conceito no seu próprio código. Você olha para uma linha e diz onde está a abstração. Você olha para outra e diz onde está o encapsulamento.

Nota. Se você responde às perguntas sem abrir o arquivo `personagem.py`, vá para a Parte B. A Parte B não exige decorar esta parte.

Todos os exemplos usam a classe que você já conhece:

```
class Personagem:
    def __init__(self, nome, vida, ataque, defesa, pocoes):
        self.nome = nome
        self.vida = vida
        self.vida_maxima = vida
        self.ataque = ataque
        self.defesa = defesa
        self.pocoes = pocoes
        self.defendendo = False
```

1. O que é Orientação a Objetos

Orientação a Objetos organiza o programa em objetos. Cada objeto guarda os próprios dados. Cada objeto executa as próprias ações.

No RPG, cada Personagem é um objeto. O Personagem guarda a própria vida. O Personagem sabe atacar, defender e usar poção.

O programa não é uma lista solta de variáveis e funções. O programa é um conjunto de objetos que trabalham juntos.

Você já fez isso. As próximas seções dão nome a quatro ideias centrais. Você já usou duas delas: abstração e encapsulamento. As outras duas aparecem na Parte B.

2. Abstração

Abstração é dar um nome simples a uma ideia complicada. Depois você usa o nome. Você não repete a ideia complicada toda vez.

Veja este método:

```
def esta_vivo(self):
    return self.vida > 0
```

No laço da batalha, você não repete a expressão `self.vida > 0` em vários lugares. Você criou o nome `esta_vivo()`. Você pergunta isso ao objeto:

```
while jogador.esta_vivo() and inimigo.esta_vivo():
    ...
```

O método `mostrar_status()` faz o mesmo com a exibição. Em vez de repetir três `print`, existe um nome para mostrar o estado.

A abstração tem duas vantagens. O código fica mais fácil de ler. A regra fica em um único lugar. Se "estar vivo" passar a significar outra coisa, você muda só dentro de `esta_vivo()`. O resto do programa continua igual.

Perguntas

1. O método `esta_vivo()` esconde a expressão `self.vida > 0`. Cite outro método do `Personagem` que esconde um detalhe atrás de um nome. Diga qual detalhe ele esconde.
2. A regra de "estar vivo" muda para: vida acima de zero e personagem não petrificado. Quantos lugares do programa você altera? Justifique.
3. Ler `if jogador.esta_vivo()`: é mais fácil que ler `if jogador.vida > 0`:. Explique por quê, sem usar código.

3. Encapsulamento

Encapsulamento tem uma regra. Cada objeto cuida do próprio estado. Quem está de fora não altera os atributos direto. Quem está de fora pede a mudança por um método. Assim o objeto garante um estado sempre válido.

Você já fez isso:

```
def receber_dano(self, quantidade):
    dano_real = quantidade - self.defesa
    if self.defendendo:
        dano_real = dano_real // 2
        self.defendendo = False
    dano_real = max(0, dano_real)
    self.vida = max(0, self.vida - dano_real)
```

O método protege quatro regras:

- o método desconta a defesa do dano;
- se o personagem defende, o dano cai pela metade e a defesa é gasta;
- o dano nunca fica negativo;
- a vida nunca fica abaixo de zero.

Ninguém de fora precisa lembrar dessas regras. Basta chamar `inimigo.receber_dano(30)`. O `inimigo` aplica o dano de forma válida.

Agora veja o atalho perigoso:

```
inimigo.vida = inimigo.vida - 30 # burla todas as regras acima
inimigo.vida = -999             # deixa o objeto num estado impossível
```

O Python permite essas linhas. Mas elas quebram o encapsulamento. A defesa é ignorada. O `defendendo` não é gasto. A vida recebe um valor sem sentido. O método existe para ninguém quebrar as regras do objeto, nem por engano.

Nota. O Python não proíbe `inimigo.vida = -999`. O encapsulamento aqui é uma decisão de projeto. Todo o programa combina em mudar a vida só por `receber_dano()` e usar `pocao()`. A disciplina vem de você, não da linguagem.

Perguntas

1. Liste todos os métodos que alteram `self.vida`. A regra do jogo permite mudar a vida fora deles? Justifique.
 2. Alguém escreveu `jogador.pocoas = jogador.pocoas + 5` no `main.py` para dar poções. Compare isso com um método `ganhar_pocao(quantidade)`. Que problema o acesso direto pode causar?
 3. O método `receber_dano()` faz `self.defendendo = False` depois de usar a defesa. O que acontece se você esquecer essa linha? Descreva o efeito no jogo, turno a turno.
 4. Cenário novo. O chefe recebe o dobro de dano enquanto defende, em vez de metade. Qual método você altera? Quais partes do programa não mudam? Justifique.
-

4. Estado e comportamento

Todo objeto tem duas partes.

O estado é o que o objeto é agora: `nome`, `vida`, `pocoas`, `defendendo`. São os atributos.

O comportamento é o que o objeto faz: `atacar`, `defender`, `usar_pocao`, `receber_dano`. São os métodos.

Os atributos guardam valores. Os métodos leem e mudam esses valores durante o jogo. O atributo `defendendo` começa em `False`. Ele vira `True` quando o personagem defende. Ele volta a `False` quando um golpe é aparado. Esse vaivém é o estado que muda com o tempo. Os métodos conduzem essa mudança.

Atenção a um ponto. Nem toda lógica do programa é comportamento do objeto. O menu de ações não é uma habilidade do personagem. O menu é tarefa do programa principal. Por isso o menu fica em `turno_do_jogador()`, uma função de fora da classe:

```
def turno_do_jogador(jogador, inimigo):
    print("1 - Atacar")
    print("2 - Defender")
    print("3 - Usar poção")
    escolha = input("Escolha: ")
    ...
```

A função decide quando uma ação acontece. O método da classe executa a ação. Essa divisão evita uma classe cheia de menu, teclado, laço e regras.

Perguntas

1. Classifique cada item como atributo ou método: `vida_maxima`, `usar_pocao`, `defesa`, `esta_vivo`, `defendendo`.
 2. Diga se cada item pertence à classe `Personagem` ou ao programa principal: o texto do menu; quanto uma poção cura; a ordem dos inimigos; a regra de que o dano não fica negativo.
 3. O atributo `defendendo` não vem por parâmetro do `__init__`. Ele nasce sempre `False`. Por que faz sentido ele não ser parâmetro do construtor?
-

5. Objeto que fala com objeto

Esta é a parte principal do RPG. Um objeto pede uma ação a outro objeto.

```
def atacar(self, alvo):
    rolagem = random.randint(1, 20)
    if rolagem < 5:
        print(self.nome, "errou o ataque")
        return
    dano = random.randint(self.ataque - 3, self.ataque + 3)
    alvo.receber_dano(dano)
    print(self.nome, "causou", dano, "de dano")
```

Na chamada `jogador.atacar(inimigo)` :

- `self` é o jogador, que ataca;
- `alvo` é o inimigo, que recebe o dano;
- `self.ataque` vem do jogador;
- o inimigo executa `receber_dano()` ;
- a vida que muda é a do inimigo.

O jogador não escreve na vida do inimigo. O jogador chama `alvo.receber_dano(dano)` . O inimigo aplica o próprio dano. Isso respeita o encapsulamento da Seção 3. Um objeto termina o próprio trabalho e pede a outro objeto que faça a parte dele.

Essa colaboração prepara a Parte B. Quando existirem vários tipos de personagem, a linha `alvo.receber_dano(dano)` continua igual. Não importa quem seja o alvo.

Perguntas

1. Em `jogador.atacar(inimigo)` , quem é `self` e quem é `alvo` ? E em `inimigo.atacar(jogador)` ?
 2. O método `atacar` chama `alvo.receber_dano(dano)` . Por que ele não escreve `alvo.vida = alvo.vida - dano` ? Use a Seção 3 na resposta.
 3. Cenário novo. Um Curandeiro precisa curar um aliado, em vez de causar dano. A assinatura `def curar(self, alvo):` ainda faz sentido? Qual método o alvo precisa ter? Escreva só o cabeçalho dos dois métodos.
 4. O que falta. Alguém escreveu `def atacar(alvo):` sem `self` , dentro da classe. O que dá errado quando o jogo chama `jogador.atacar(inimigo)` ? Justifique.
-

Sequência de estudo

1. Leia uma seção. Antes das perguntas, diga o conceito com suas palavras.
2. Responda às perguntas com o arquivo `personagem.py` aberto, não com o gabarito.
3. Onde a pergunta pede código, escreva o código. Escreva ao menos o cabeçalho.
4. No final, complete a frase sem consultar nada: "no meu RPG, abstração é ____, encapsulamento é ____, estado é ____, comportamento é ____."
5. Quando você completar a frase com exemplos do seu código, vá para a Parte B.

Como saber se você entendeu

Você não precisa decorar definições. Você entende quando aponta no seu código onde cada conceito acontece. Você entende quando explica por que ele está ali. O nome é só a etiqueta. O importante é reconhecer a coisa.